

Stammtisch: Teaching a Computer to Play Schieber Jass

Learning from 1.8 million human games, searching over hidden cards,
and coaching human players

Dario Hug

Progress report, 27 September 2026

Abstract

Jass is Switzerland’s national card game. Its most popular variant, *Schieber*, is played by two teams of two, and every player sees only their own nine cards. This makes Jass much harder for a computer than chess: the program has to reason about cards it cannot see, cooperate with a partner without talking, and cope with luck. We describe *Stammtisch*, a Jass program built in three layers. (1) A fast, exactly verified game engine that replays all 1’821’878 human games of a public Swisslos dataset without a single rule violation. (2) A neural network that learned to play like a typical human by studying 65.6 M human card plays; it predicts the card a person will play 79.9% of the time. (3) A search that imagines many possible distributions of the hidden cards, weights them by how well they explain what the other players did, and plays each one out to see which card works best. In matches with identical cards for both sides, the search beats the human-like network by $+28.4 \pm 7.8$ points per round. We also turned the engine into a *coach*: it reviews recorded games and tells a player, with an honest error bar, where a different card would have been better. This report is written for readers who are new to machine learning; boxes marked “*In plain words*” explain the key ideas.

1 Introduction

Computers beat the best humans at chess (1997), Go (2016) and poker (2019). Card games like Jass are still an open challenge for three reasons:

- **Hidden information.** You see 9 of 36 cards. The other 27 can be split among the three other players in about $2.3 \cdot 10^{12}$ ways. A good player constantly guesses who holds what.
- **Teamwork without talking.** You win or lose with your partner, but you may not talk. Information flows only through which cards you play (for example, “schmieren”: giving your partner a valuable card when their trick is safe).
- **Luck.** A strong player with bad cards loses a single round. Measuring who is better needs many rounds and careful statistics.

Earlier work on Jass (Section 2.4) reached the level of good amateurs. Our goals are (a) a program that beats typical human players and (b) a coach that helps a human player improve. This report makes the following contributions:

1. A fast and **verified** Schieber engine with official Swisslos scoring (Weis, Stöck, multipliers) (Section 4.1).
2. **Imitation learning** on 1’821’878 human games: neural networks that play the cards and choose trump like humans do (Section 4.3).
3. A **search** that samples the hidden cards, uses the networks to judge how plausible each sample is, and solves the end of each round exactly (Sections 4.4–4.5).
4. A careful **evaluation** with duplicate deals and confidence intervals (Sections 5–6).
5. A **coach** that reviews a player’s own games (Section 4.7).

2 Background

2.1 Schieber in one paragraph

Each player gets 9 cards from a 36-card deck (four suits: Eichel, Rosen, Schilten, Schellen; ranks 6 to Ass). The *forehand* chooses the trump mode: one of the four suits, *Obenabe* (no trump, high cards win) or *Undeufe* (no trump, low cards win), or they *push* (“schieben”) the choice to their partner. Then nine tricks are played: everyone must follow the suit that was led, but may always play a trump instead; the highest card wins the trick. Among trumps the Under (Puur, 20 points) and the Nell (9, 14 points) are the strongest. All cards together are worth 152 points, plus 5 for the last trick; a team that wins all nine tricks gets 257 (a *match*). Swisslos multiplies the points by 1 (Eichel, Rosen), 2 (Schilten, Schellen) or 3 (Obenabe, Undeufe) and adds bonus points for card combinations (*Weis*) and for holding King and Ober of trump (*Stöck*, 20 points).

	← stronger								
trump suit	Under	Nell	Ass	König	Ober	Banner	8	7	6
points	20	14	11	4	3	10	0	0	0
other suits	Ass	König	Ober	Under	Banner	9	8	7	6
points	11	4	3	2	10	0	0	0	0

Figure 1: Card strength (left = strongest) and points. In the trump suit the Under and the 9 (Nell) jump to the top.

2.2 How computers play games

In plain words: the game tree and minimax

A game can be drawn as a *tree*: every position branches into the possible moves, down to the end of the game. If you could see the whole tree, the best move is found by *minimax*: assume your side picks the move that is best for you, the opponents pick the move that is worst for you, and work backwards from the end. *Alpha-beta pruning* skips branches that cannot change the result, which makes this much faster.

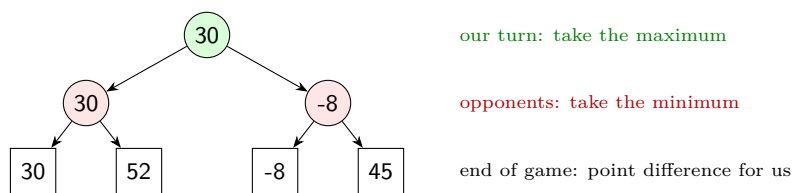


Figure 2: Minimax on a tiny tree. Our left move guarantees 30 points, the right move only -8 , so we play left.

Minimax needs to know all cards. In Jass we only know our own. The classic trick for card games is *Perfect Information Monte Carlo* (PIMC, used for Bridge [2] and Skat [4]): **imagine** a possible deal of the hidden cards, solve it as if all cards were open, repeat for many imagined deals, and pick the move that is best on average (Figure 3).

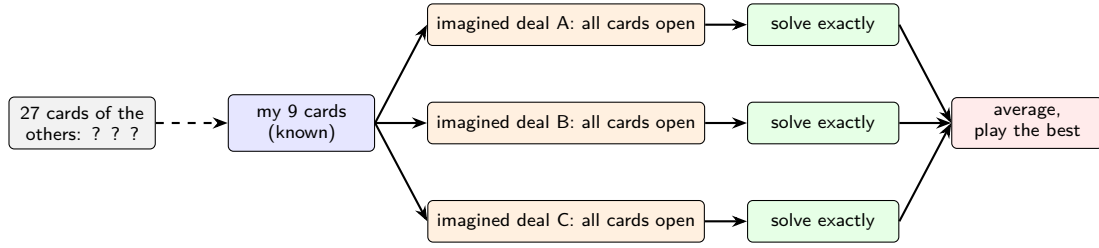


Figure 3: The idea behind PIMC: fill in the unknown cards in many plausible ways, solve each imagined deal, and average the results.

2.3 Machine learning in a nutshell

In plain words: what a neural network is and how it learns

A *neural network* is a function with many adjustable numbers (“weights”). It takes a list of numbers as input (here: which cards I hold, which cards were played, what trump is) and outputs a score for every possible card. *Training* means showing it millions of examples (“in this situation, the human played the Rosen Ass”) and nudging the weights a tiny bit after each example so that the correct answer gets a higher score next time. To check that the network learned something general instead of memorizing, we test it on *validation* games it never saw during training.

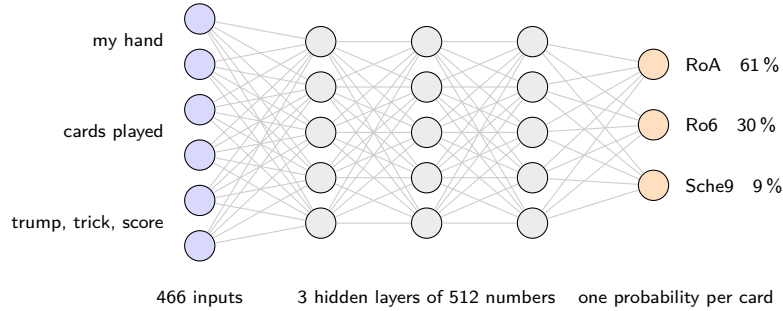


Figure 4: The card network. It reads the situation as 466 numbers and outputs a probability for each legal card. The real network is much larger than drawn.

2.4 Related work

Jass has been studied mainly at Swiss universities. The Lucerne University of Applied Sciences (HSLU) teaches a course *Deep Learning for Games* built around Jass and provides the **jass-kit** software [7] and the Swisslos dataset we use. A survey from the University of Fribourg [1] compares approaches, and theses at the University of Bern tried AlphaZero-style learning and game-theoretic methods; the best programs reached the level of strong amateurs. Our approach combines ideas from Bridge (PIMC [2, 3]), Skat (learning about the opponents’ cards from their play [4]) and AlphaZero-style self-improvement [6].

3 Data

The dataset contains 1’821’878 Schieber rounds played online at Swisslos between 2017-10-08 and 2018-04-15 by 4’608 registered players. Each round lists all nine tricks, so we can reconstruct all four hands. This gives 65.6 M card decisions and about 3 million trump decisions, each with the correct “answer” (what the human did). We keep the most recent 5 % of games (by date) apart as validation data.

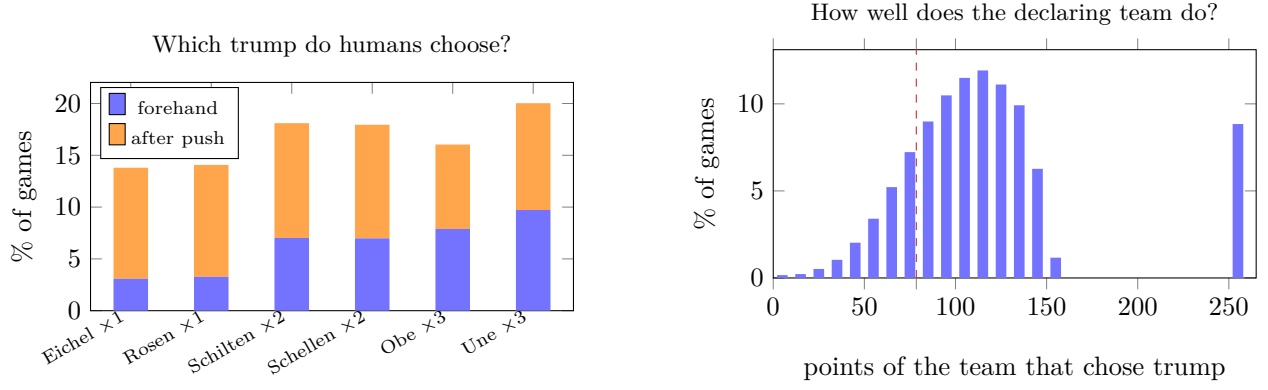


Figure 5: Left: players push in 62.0% of rounds and prefer the high-multiplier modes when they choose directly. Right: the team that chose trump wins 81.0% of rounds (dashed line: half of 157); 9.0% of rounds are matches.

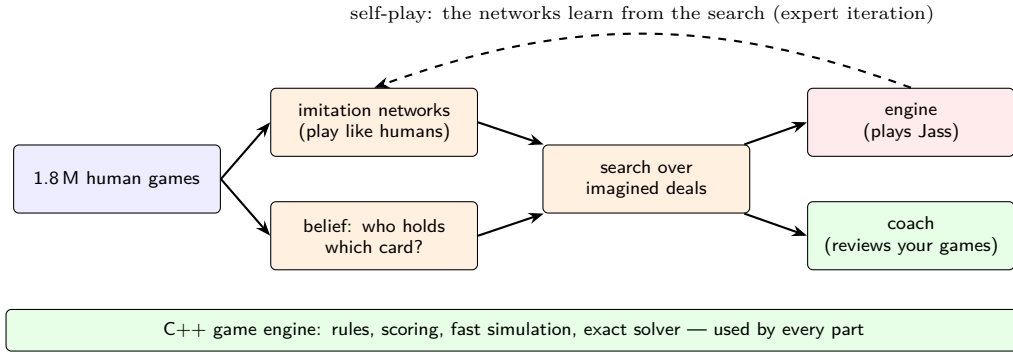


Figure 6: Overview. Human games teach the networks; the networks guide the search; the search plays, coaches, and produces new training data for the networks.

4 Method

4.1 Game engine and simulator

The engine stores a hand as a 64-bit number in which bit i says whether card i is present. Checking which cards may legally be played then takes a handful of processor instructions. On a laptop (14 threads) the simulator plays 6.1 M rounds per second. A live dashboard shows throughput, results, CPU load and temperature while it runs.

Verification. Bugs in the rules would silently poison everything built on top. We therefore replayed every one of the 1’821’878 human games: all 65.6 M card plays are legal and every trick winner and score matches the log. We also compared our rules with HSLU’s `jass-kit` on 119 073 random positions. The only disagreements (0.15%) turned out to be a bug in `jass-kit`, which in rare cases allows “under-trumping”.

4.2 Exact solver for open cards

If all cards were visible, the best play could be computed exactly with minimax and alpha-beta pruning. In card games this is called a *double-dummy* solver. Three standard tricks make ours fast: it remembers positions it has already solved (a *transposition table*), treats cards that are interchangeable (e.g. the 7 and the 6 when nothing lies between them) as one, and narrows in on the exact score with a sequence of yes/no questions (*MTD(f)* [5]). It agrees with a brute-force search on 2 560 random endgames. Its cost grows about tenfold per extra trick (Figure 8), so we use it for the last $k = 5$ tricks only.

4.3 Imitation learning: playing like a human

The *card network* (Figure 4) receives what the player can see, encoded as 466 zeros and ones: own hand, cards played by each player, the cards in the current trick, trump, who chose trump, which suits a player has

shown to be out of, and the score. It outputs a probability for each of the 36 cards; illegal cards are masked out. It was trained on 123M examples, about 40 minutes on the laptop’s processor. A second, smaller *trump network* looks at the 9 cards of a hand and predicts the trump choice (or push).

In plain words: why imitate humans at all?

Imitation gives us three things at once: a decent player without any hand-written rules; a *model of human behaviour* (“how likely would a person play this card with that hand?”), which we need for the belief in Section 4.5; and a realistic *sparring partner* to measure our engine against.

4.4 Search over imagined deals

To choose a card, the engine repeats $n = 32$ times: (1) imagine a deal of the hidden cards that agrees with everything the player has seen (for example, a player who did not follow suit has no card of that suit left), (2) for each legal card, play the rest of the round: the human-like network plays for everybody until 5 tricks are left, then the exact solver finishes the round, (3) note the resulting point difference. The card with the best average wins. The trump choice is made by the trump network: a search over all six modes was not better (Section 6).

4.5 Belief: learning from the other players’ behaviour

Uniformly imagined deals ignore valuable hints. If your opponent chose Rosen as trump, they probably hold the Rosen Under; if your partner pushed, their hand is probably weak. We use the networks to ask, for each imagined deal: *how likely would the other players have made exactly the choices we saw, if this were the true deal?* Deals that explain the observed play well get a higher weight (this is Bayes’ rule). In practice we imagine $4n$ deals, weight them by this likelihood raised to the power $b = 0.5$ (a softening that keeps the method robust when people play differently from the model), and keep n of them in proportion to their weights.

In plain words: a small example of the belief

Imagine two possible worlds. In world A your left opponent holds the Rosen Under, in world B they do not. They chose Rosen as trump. Suppose the trump network says: with the Under, a human picks Rosen 70 % of the time; without it, only 5 %. Then world A becomes 14 times more likely than before, and the search spends most of its effort on deals like A.

4.6 Expert iteration: learning from the search

The search plays better than the network it uses. So we let the search play 8 000 rounds against itself (one hour on the laptop) and trained a copy of the network on the search’s 288 000 decisions, mixed with 25 % human data so it does not forget what it knew. Its agreement with the search on unseen positions rose from 54.7 % to 61.4 % of the real choices. This is the idea behind AlphaZero [6]: the search improves the network, and a better network improves the search. We call the result **net:strong**.

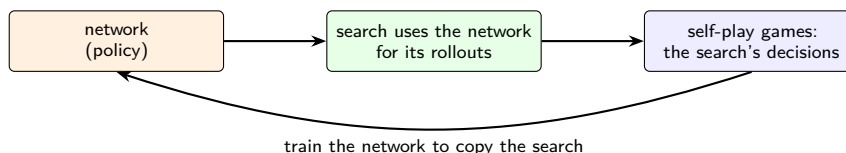


Figure 7: Expert iteration: a loop in which the network and the search improve each other. We have run one round.

4.7 The coach

A recorded game is typed in a simple text format (players, dealer, trump, and the tricks, with German or French card names). For every decision the coach runs the same search *from that player’s point of view*, using only what they could see, and reports how many points each alternative card was expected to gain or lose. Because the search is based on random samples, every number comes with a standard error; only

losses clearly larger than this noise are called mistakes. The coach also gives a one-line reason taken from the actual trick (e.g. “schmieren: RoB gives your team 10 more points”) and, in the last tricks, the exact best card with all hands open. The same evaluation is available live (“which card should I play now?”) and while playing against the engine.

5 How we measure strength

In plain words: fair comparisons in a game of luck

Duplicate deals. Every deal is played twice: once agent A holds the North/South cards, once the East/West cards. Good or bad cards then cancel out, and mostly skill remains.

Confidence interval. We report results like “ $+28 \pm 8$ points per round”: the true advantage lies within this range with 95 % probability. If the range does not include 0, the difference is *significant*.

Elo. Like in chess, a rating derived from how often one agent wins rounds against another; 100 points more means winning about 64 % of rounds against that opponent.

We compare these agents: **random** (random legal cards), **heuristic** (hand-written rules, e.g. lead your highest card, give points to a winning partner), **net** (imitation of humans), **net:strong** (after expert iteration), **pimc** (search with belief), and variants of the search. All numbers are in *slate points* (with Swisslos multipliers, Weis and Stöck), per round.

6 Results

6.1 Speed

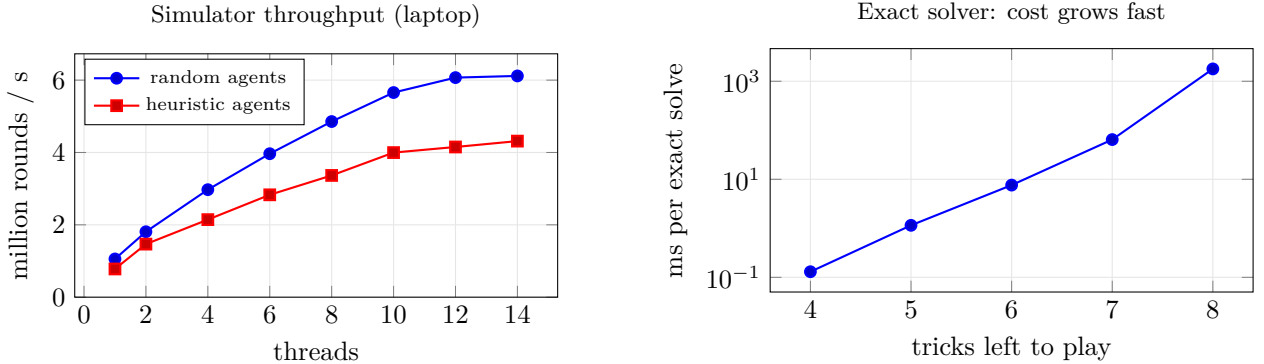
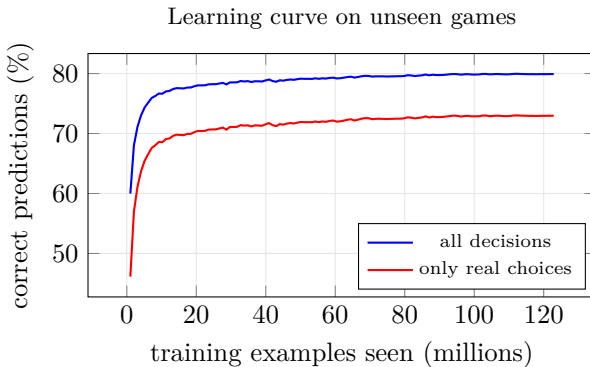


Figure 8: Left: the simulator scales over the laptop’s cores. Right: solving a position exactly takes about a millisecond with 5 tricks left but seconds with 8, which is why the search solves only the last 5 tricks exactly.

6.2 How well does the network imitate humans?



The network predicts the human’s card in 79.9 % of validation decisions and in 73.0 % of the decisions where more than one card was legal. The trump network matches 80.1 % of human trump calls. The curve flattens after about 60 million examples: more of the same data would not help much.

A perfect score is impossible: two humans in the same situation often play differently, and many decisions are close.

6.3 Who plays best?

#	agent	Elo
1	pimc:roll=strong	1654
2	pimc	1651
3	net:strong	1628
4	net	1620
5	heuristic	1500

A	B	deals	A–B per round	±
heuristic	net	40000	-95.6	1.2
heuristic	net:strong	40000	-96.8	1.2
heuristic	pimc	400	-107.6	11.0
heuristic	pimc:roll=strong	400	-115.6	11.5
net	net:strong	40000	-3.3	0.6
net	pimc	400	-28.4	7.8
net	pimc:roll=strong	400	-27.3	6.8
net:strong	pimc	400	-20.3	7.0
net:strong	pimc:roll=strong	400	-21.6	7.5
pimc	pimc:roll=strong	400	-2.2	6.6

The main findings:

- **Learning beats hand-written rules.** The imitation network beats the heuristic by almost 100 points per round.
- **Search beats imitation.** The search agent `pimc` beats the human-like network by $+28.4 \pm 7.8$ points per round. This is our best estimate of its advantage over a typical online player, with the caveat that the network is only a stand-in for real humans.
- **The belief matters.** Against the human-like imitation policy, search with belief scores $+28.4 \pm 7.8$ points per round, the same search without belief $+15.0 \pm 7.4$.
- **Expert iteration works, a little.** After one round, `net:strong` beats the human imitation by $+3.3 \pm 0.6$ points per round (significant over 40 000 deals). Using it inside the search did not yet make a measurable difference.

What did not help. Each variant played 600 duplicate deals against the earlier default search (32 samples, trump by searching 16 deals per mode, exact for 5 tricks, belief 0.5); positive = variant better:

variant	change	points / round	±
<code>pimc:b=1</code>	belief strength 1 instead of 0.5	-6.9	7.9
<code>pimc:t=0</code>	trump by the policy instead of 16-deal search	+4.8	7.7
<code>pimc:k=6</code>	exact for the last 6 tricks instead of 5 (2.3× slower)	+3.2	8.6
<code>pimc:b=0</code>	no belief (uniform sampling)	-9.5	10.2

None of these differences is significant. Interestingly, choosing trump the way humans do is as good as searching over all six modes, and much faster, so it became the default.

6.4 The coach in action

Figure 9 shows a review of an example round. Each card played by the reviewed team is marked green (good), yellow/orange/red (increasingly costly) or grey (difference within the sampling noise). As a sanity check, the coach judged random play worst, the heuristic better and the human-like network best, as it should.

7 Discussion and limitations

- **Humans are not networks.** Our strongest evidence compares against a network that imitates average Swisslos players. Real people plan ahead, use conventions with their partner and adapt. The next step is to play against real people, for example in the HSLU tournament system, for which we already built an adapter.
- **Imagining is not knowing.** PIMC solves each imagined deal as if everyone could see all cards. It can therefore overrate plays that only work with perfect knowledge (“strategy fusion” [3]). The rollouts with the human-like network soften this, but do not remove it.
- **Small experiments.** Many of our comparisons used 400–600 deals, which can only detect differences of about 8 points per round. Smaller improvements need more compute.
- **Data licence.** The Swisslos data comes from public course material; we have asked HSLU for permission to use it and do not publish the data or the trained networks until then.

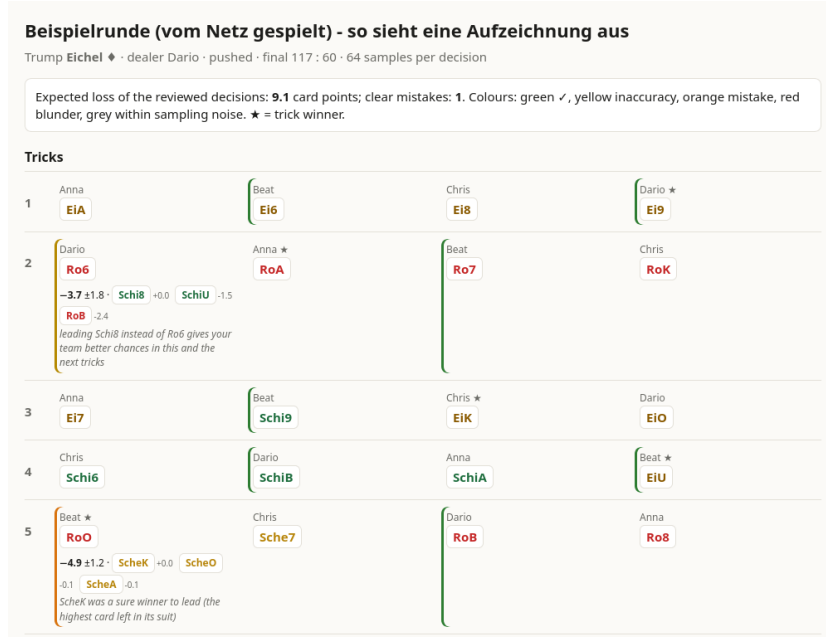


Figure 9: HTML review of a round. In trick 2 the coach suggests leading a different card than Ro6; the difference is within the sampling noise, so it is shown in grey rather than as a mistake.

8 Conclusion and next steps

With a verified engine, imitation of 1.8 million human games, and a search that reasons about hidden cards and about what the other players' choices reveal, Stammtisch beats a human-like opponent by $+28.4 \pm 7.8$ points per round, and it can show a player where their own games went wrong. Next we plan to (1) run more rounds of expert iteration with a larger compute budget, (2) replace the rollouts by a network that directly predicts the outcome of a position (a *value network*), (3) speed up the exact solver, and (4) test the engine and the coach against real players.

Glossary

Validation data: examples held back from training to test the network honestly. **Policy:** a rule (here: a network) that picks a move. **Rollout:** playing a round to the end with a policy to see what happens. **Double dummy:** solving a deal with all cards open. **PIMC:** imagine the hidden cards, solve each imagined deal, average. **Belief:** how likely each distribution of the hidden cards is, given what we saw. **Expert iteration:** training a network to copy a stronger search. **Duplicate deals:** playing each deal twice with swapped seats to cancel luck. **Significant:** unlikely to be a coincidence (here: the 95 % range excludes zero).

References

- [1] J. Niklaus, M. Alberti, V. Pondenkandath, R. Ingold, M. Liwicki. *Survey of Artificial Intelligence for Card Games and Its Application to the Swiss Game Jass*. Swiss Conference on Data Science, 2019.
- [2] M. L. Ginsberg. *GIB: Imperfect information in a computationally challenging game*. Journal of Artificial Intelligence Research, 2001.
- [3] J. Long, N. Sturtevant, M. Buro, T. Furtak. *Understanding the success of perfect information Monte Carlo sampling in game tree search*. AAAI, 2010.
- [4] M. Buro, J. Long, T. Furtak, N. Sturtevant. *Improving state evaluation, inference, and search in trick-based card games*. IJCAI, 2009.
- [5] A. Plaat, J. Schaeffer, W. Pijls, A. de Bruin. *Best-first fixed-depth minimax algorithms*. Artificial Intelligence, 1996.
- [6] D. Silver et al. *A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play*. Science, 2018.
- [7] HSLU, *jass-kit-py*, <https://github.com/thomas-koller/jass-kit-py>; Swisslos Schieber rules, <https://www.swisslos.ch/de/jass/informationen/jass-regeln/schieber-jass.html>.